EPIS · UNIVERSIDAD NACIONAL DE SAN AGUSTÍN · 2026

Verificación formal de coherencia en políticas ABAC

Garantías matemáticas para evitar bloqueos indebidos y filtraciones de datos en sistemas de autorización

Yenaro Joel Noa Camino
Ingeniería de Sistemas · 4to año

Luis Gustavo Sequeiros Condori
Ingeniería de Sistemas · 4to año

Christian Raul Mestas Zegarra
Ingeniería de Sistemas · 4to año

Mariel Alisson Jara Mamani
Ingeniería de Sistemas · 4to año

¿POR QUÉ ESTE PROBLEMA ES CRÍTICO EN EL MUNDO REAL?

El dolor en entornos críticos: salud y organizaciones públicas

La administración manual de directivas de acceso introduce fallas lógicas que comprometen vidas y datos reservados

Escenario de Urgencia Hospitalaria

En un centro médico, las autorizaciones dependen de turnos, guardias y condiciones de emergencia. Una contradicción inadvertida entre directivas puede bloquear el acceso de un cirujano a un expediente clínico crítico en plena intervención quirúrgica.

Impacto Operativo Crítico

Denegaciones indebidas que paralizan atenciones de urgencia o permisos involuntarios que filtran datos confidenciales.

La Causa Raíz: Crecimiento y Explosión Combinatoria

El estándar NIST SP 800-162 formula directivas sobre cuatro familias de atributos: sujeto, recurso, acción y entorno. Con múltiples atributos discretos, el espacio cartesiano de solicitudes posibles supera con rapidez las decenas de miles de combinaciones.

Insuficiencia del Muestreo Empírico

Las pruebas tradicionales por muestreo dejan zonas ciegas en el espacio de solicitudes. Solo la verificación deductiva provee garantías totales.

NIST SP 800-192 formaliza la necesidad de verificar exhaustivamente conjuntos de políticas de acceso.

¿QUÉ PUEDE SALIR MAL AL REDACTAR REGLAS DE ACCESO?

Tres fallas silenciosas que los tests tradicionales no logran detectar

Clasificación canónica de defectos lógicos en políticas de control de acceso

ANOMALÍA 01

Contradicción

Una misma solicitud concreta activa simultáneamente una regla de Permiso y una de Denegación. Al carecer de resolución matemática, el veredicto en producción queda a merced del orden de evaluación o la implementación del motor.

ANOMALÍA 02

Regla Insatisfacible

Directiva con condiciones mutuamente incompatibles sobre los rangos del esquema. La regla no puede activarse bajo ninguna combinación válida de atributos, constituyendo código muerto dentro de la política.

ANOMALÍA 03

Sombreado

Directiva completamente cubierta y anulada por reglas preexistentes de mayor generalidad. No existe ninguna solicitud en el sistema donde esta directiva decida en solitario, acumulando redundancia oculta.

Nuestro verificador detecta las tres anomalías y sintetiza un testigo diagnóstico con la tupla mínima infractora.

NUESTRA PROPUESTA: SOLUCIONAR LA CAUSA RAÍZ ANTES DE EJECUTAR

Seguridad por construcción: certificar antes de desplegar

Demostración matemática en tiempo de diseño frente a la fragilidad del análisis en tiempo de ejecución

ENFOQUE CONVENCIONAL EN PRODUCCIÓN

Invocación Externa a Solucionadores SMT

Los esquemas industriales tradicionales traducen consultas a solucionadores SMT en vivo durante cada petición o mediante análisis a posteriori sobre políticas ya desplegadas.

- **Sobrecarga de Latencia**
  Retardo significativo en servidores de autorización en tiempo real.
- **Riesgo de Timeouts**
  Tiempos de espera indefinidos o agotamiento de memoria en el resolvedor.

NUESTRA PROPUESTA FORMAL

Núcleo Compilado y Guardas Inductivas

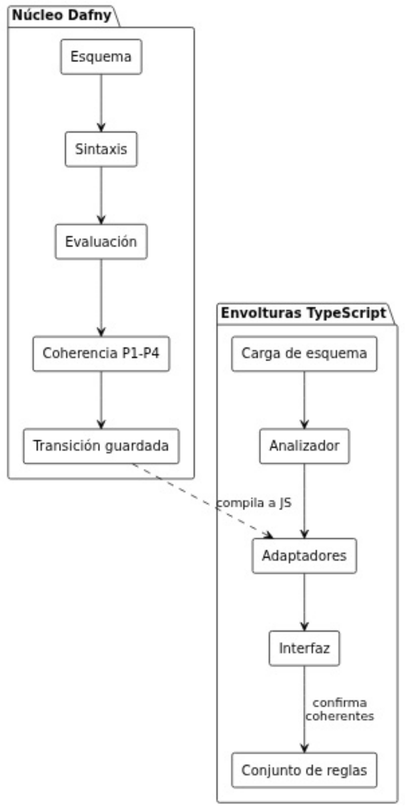
Los motores SMT Z3 y cvc5 operan únicamente en tiempo de desarrollo para certificar los lemas deductivos en Dafny. El artefacto final es JavaScript puro ejecutándose sin motores externos.

- **Cero Motores en Producción**
  Código determinista ejecutado directamente en Node.js o el navegador.
- **Preservación Inductiva**
  Cada mutación verifica la coherencia antes de confirmar el nuevo estado.

¿CÓMO CONECTAMOS LA TEORÍA FORMAL CON UNA APLICACIÓN REAL?

Arquitectura del sistema en tres capas integradas

Aislamiento estricto y dependencias unidireccionales entre especificación formal y entorno anfitrión

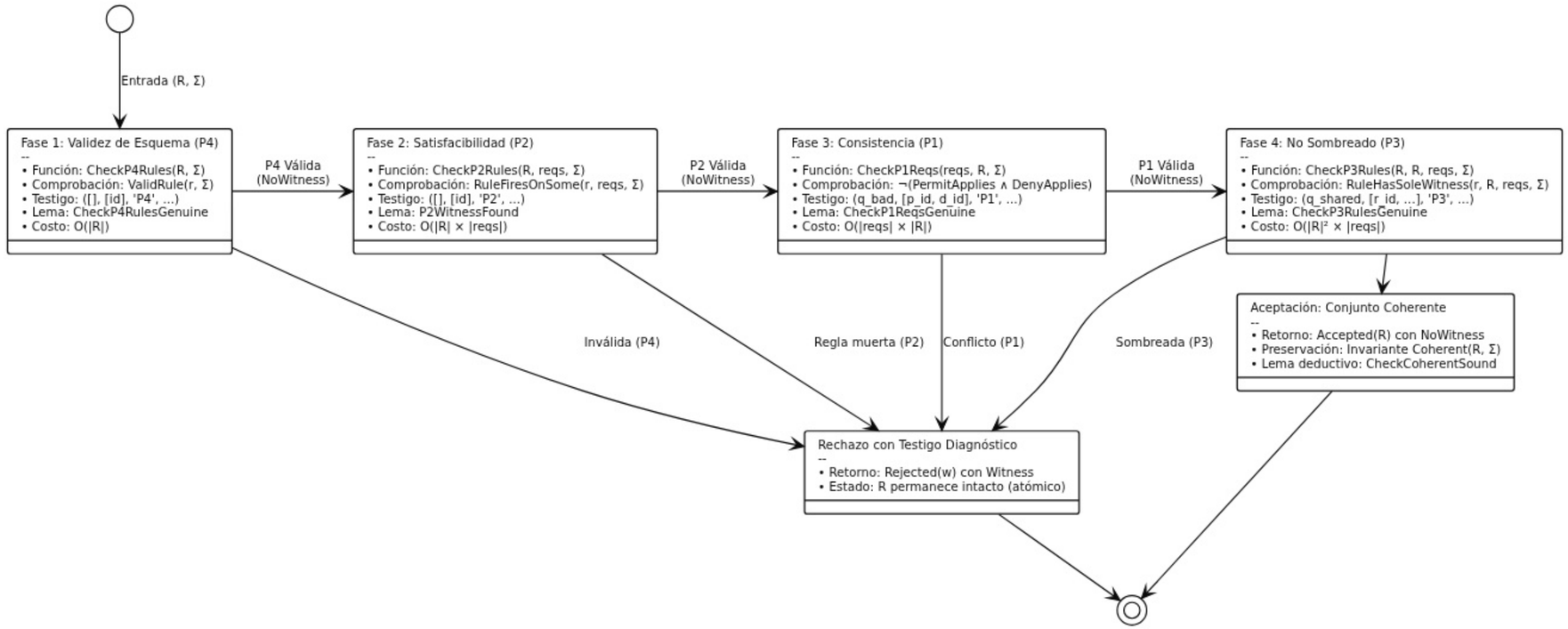


- **1. Núcleo Formal en Dafny**
  Módulos de esquema, sintaxis abstracta, semántica denotacional, comprobación de coherencia y operaciones de transición guardada con 7 lemas probados.
- **2. Código JavaScript Extraído**
  Compilación directa de las funciones funcionales puras de Dafny a JavaScript estándar, garantizando equivalencia semántica total y ejecución inmediata.
- **3. Frontera Defensiva en TypeScript**
  Analizador sintáctico estricto, cargador de esquema, adaptador de tipos, formateador de diagnósticos y distribución en el paquete npm @grupo1/abac-verifier.

¿CÓMO ANALIZAMOS CADA REGLA SIN DEJAR NINGÚN CASO AL AZAR?

Cuatro etapas continuas de verificación con respuesta inmediata

Orden secuencial de comprobación P4 a P3, lemas deductivos asociados y emisión de testigos mínimos



Definición canónica: Coherent(R) = P1(R) ∧ P2(R) ∧ P3(R) ∧ P4(R) con emisión del primer testigo mínimo en orden secuencial.

GARANTÍA MATEMÁTICA DE EVOLUCIÓN: CADA ADICIÓN PRESERVA EL SISTEMA

El núcleo en Dafny: guardas atómicas y lemas de preservación

Demostración matemática automatizada mediante los solucionadores SMT Z3 y cvc5

Transition.dfy — Operación Atómica y Demostración Inductiva

```
function TryAdd(R: Ruleset, r: Rule, s: Schema): AddResult {
  var w := CheckCoherent(R + [r], s);
  if w.NoWitness? then Accepted(R + [r])
  else Rejected(w.wit)
}

lemma AddPreserves(R: Ruleset, r: Rule, s: Schema)
  ensures match TryAdd(R, r, s)
    case Accepted(rs) ⇒ Coherent(rs, s)
    case Rejected(_) ⇒ true
{
  var w := CheckCoherent(R + [r], s);
  if w == NoWitness {
    CheckCoherentSound(R + [r], s);
  }
}
```

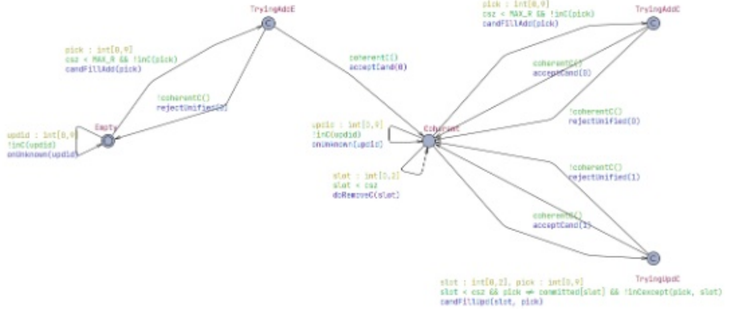
Siete Lemas Deductivos Demostrados

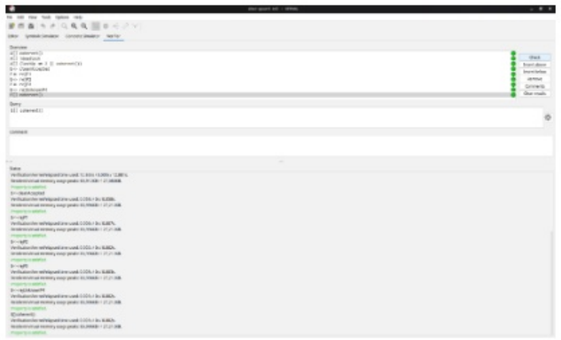
- **EmptyCoherent**
  El conjunto vacío de directivas es vacuamente coherente (caso base).
- **AddPreserves & UpdatePreserves**
  Toda confirmación aceptada preserva la coherencia global por construcción.
- **RemovePreservesCoherent**
  La eliminación de directivas mantiene la coherencia por monotonicidad.
- **WitnessSoundness & WitnessCompleteness**
  Todo rechazo genera un testigo auténtico de la propiedad vulnerada.

VERIFICANDO EL PROTOCOLO DEL ADMINISTRADOR ANTE EVENTOS CONCURRENTES

Model checking en UPPAAL: ausencia de bloqueos e invariancia estricta

Validación exhaustiva de 9 consultas en lógica TCTL sobre el autómata de transición del administrador





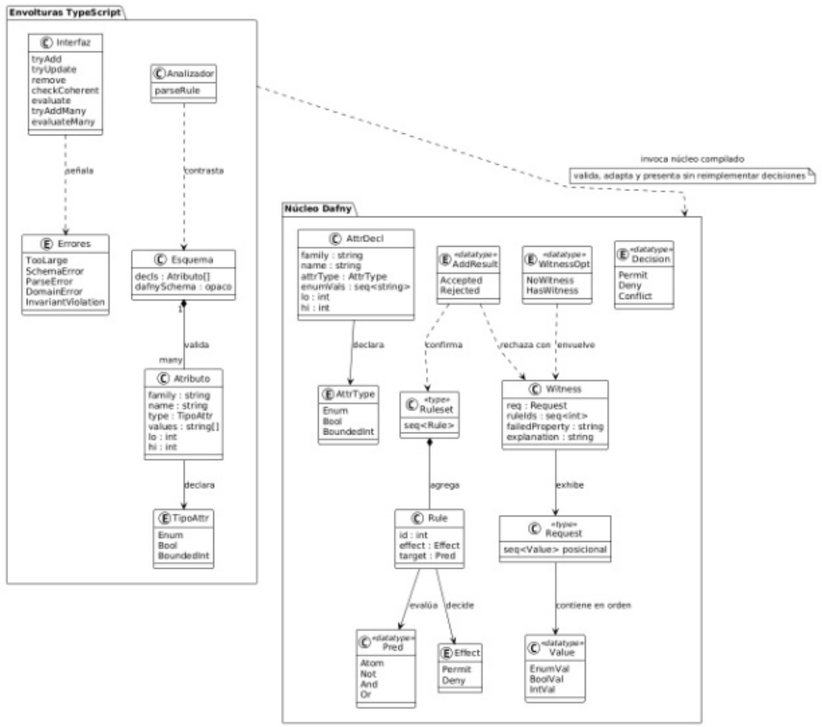
Resultados de Comprobación Temporal

- **Invariante Global: A[] coherent()**
  Todo estado alcanzable en el protocolo de administración es matemáticamente coherente.
- **Ausencia de Interbloqueos: A[] !deadlock**
  El autómata nunca queda atrapado en un estado sin transición válida disponible.
- **Alcanzabilidad de Rechazos**
  Confirmación de alcanzabilidad de cada clase de rechazo diagnóstica (rejP1, rejP2, rejP3).

HACIENDO LA VERIFICACIÓN ACCESIBLE PARA CUALQUIER DESARROLLADOR

Frontera defensiva en TypeScript: paquete npm

Validación sintáctica estricta, adaptación de tipos y explicaciones comprensibles sin duplicar lógica formal



Componentes de Integración

- **Analizador Sintáctico (parser.ts)**
  Verifica identificadores, operadores y literales directamente contra el esquema cargado antes de ingresar al núcleo.
- **Adaptadores Seguros (wrappers.ts)**
  Mapea estructuras de alto nivel hacia funciones Dafny sin recalcular ninguna decisión lógica en TypeScript.
- **Explicaciones Legibles (explain.ts)**
  Traduce la tupla mínima formal a diagnósticos en lenguaje natural para los administradores del sistema.

EL SISTEMA EN FUNCIONAMIENTO ANTE SITUACIONES REALES DE USO

Detección inmediata de conflictos y emisión de testigos mínimos

Comportamiento en terminal ante directivas conflictivas e incorporación atómica en casos de urgencia

```
Status
A[] coherent()
Verification/kernel/elapsed time used: 16.748s / 0.02s / 16.787s.
Resident/virtual memory usage peaks: 58,724KB / 101,864KB.
Property is satisfied.
A[] !deadlock
Verification/kernel/elapsed time used: 28.719s / 0.02s / 28.764s.
Resident/virtual memory usage peaks: 71,276KB / 114,440KB.
Property is satisfied.
A[] (lastOp != 2 || coherent())
Verification/kernel/elapsed time used: 12.866s / 0.005s / 12.881s.
Resident/virtual memory usage peaks: 83,912KB / 127,080KB.
Property is satisfied.
E<> cleanAccepted
Verification/kernel/elapsed time used: 0.058s / 0s / 0.058s.
Resident/virtual memory usage peaks: 83,996KB / 127,212KB.
Property is satisfied.
E<> rejP1
Verification/kernel/elapsed time used: 0.006s / 0s / 0.007s.
Resident/virtual memory usage peaks: 83,996KB / 127,212KB.
Property is satisfied.
```

```
E<> rejP2
Verification/kernel/elapsed time used: 0.002s / 0s / 0.002s.
Resident/virtual memory usage peaks: 83,996KB / 127,212KB.
Property is satisfied.
E<> rejP3
Verification/kernel/elapsed time used: 0.003s / 0s / 0.003s.
Resident/virtual memory usage peaks: 83,996KB / 127,212KB.
Property is satisfied.
E<> rejUnknownP4
Verification/kernel/elapsed time used: 0.001s / 0s / 0.002s.
Resident/virtual memory usage peaks: 83,996KB / 127,212KB.
Property is satisfied.
E[] coherent()
Verification/kernel/elapsed time used: 0.001s / 0s / 0.002s.
Resident/virtual memory usage peaks: 83,996KB / 127,212KB.
Property is satisfied.
```

Evidencia de Ejecución Real

- **Estructura del Testigo Diagnóstico**
  Ante una incoherencia, la guarda emite la solicitud exacta en disputa y los IDs de directivas en pugna, preservando el conjunto intacto.
- **Excepción de Emergencia Médica**
  La incorporación de directivas clínicas para urgencias fuera de horario comercial es aceptada atómicamente tras certificar no-sombreado.

EVIDENCIA CUANTITATIVA Y EVALUACIÓN EN ESCENARIOS REALES

Validación experimental y banco de pruebas canónico

Certificación sobre escenarios reales de uso y evaluación de desempeño instantáneo

7	9 / 9	0	100%
Lemas Demostrados en Dafny	Consultas UPPAAL Satisfechas	Interbloqueos en el Protocolo	Reproducción de Testigos

Banco de Pruebas Canónico

Suite automatizada con escenarios estructurados: clean, contradictory, unsatisfiable, shadowed y emergency-override, certificando la fidelidad del compilado.

Rendimiento Inmediato y Cero Sobrecosto

La compilación a JavaScript puro ejecuta la comprobación de coherencia en submilisegundos, permitiendo validar directivas dinámicas en pipelines de despliegue y servidores sin penalizar la latencia operativa.

IMPACTO Y VALOR PARA LA INGENIERÍA DE SISTEMAS

Certeza matemática en sistemas de información reales

Demostramos que los métodos formales rigurosos se integran armónicamente con la ingeniería web moderna: generamos un componente certificado que garantiza seguridad por construcción sin penalizaciones de latencia en producción.

PAQUETE NPM DISTRIBUIDO	REPOSITORIO DE CÓDIGO ABIERTO
`@grupo1/abac-verifier`	github.com/microuni-unsa/afve-teo-turno-c-grupo-1-final